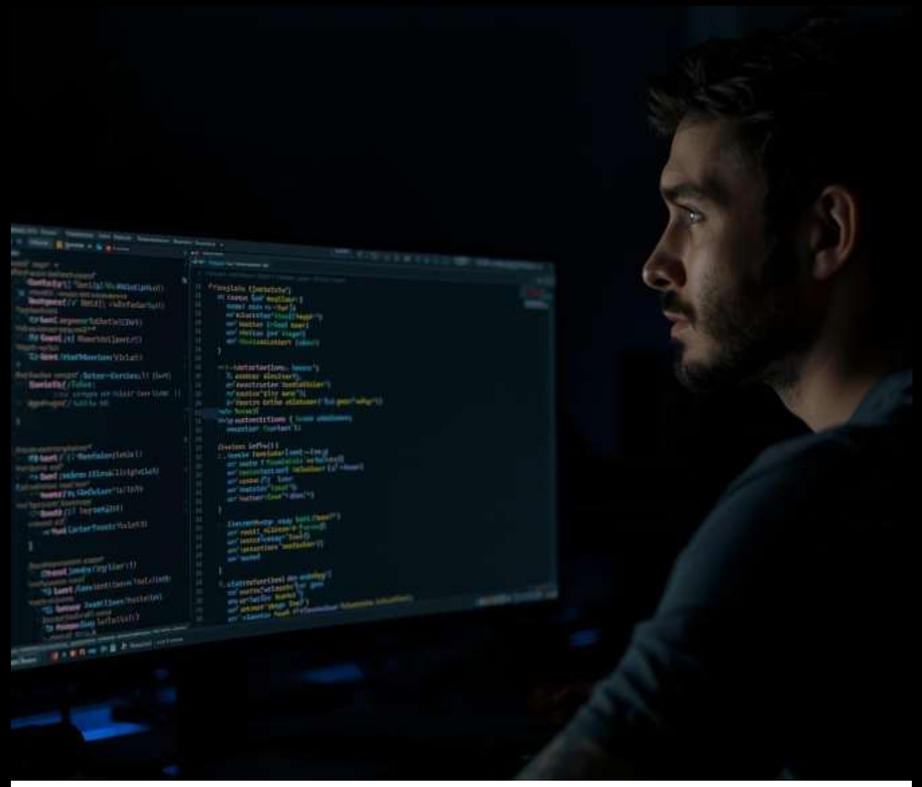
University
Project

Builder Design Pattern

PC Customization
System Case Study



Builder Design Pattern

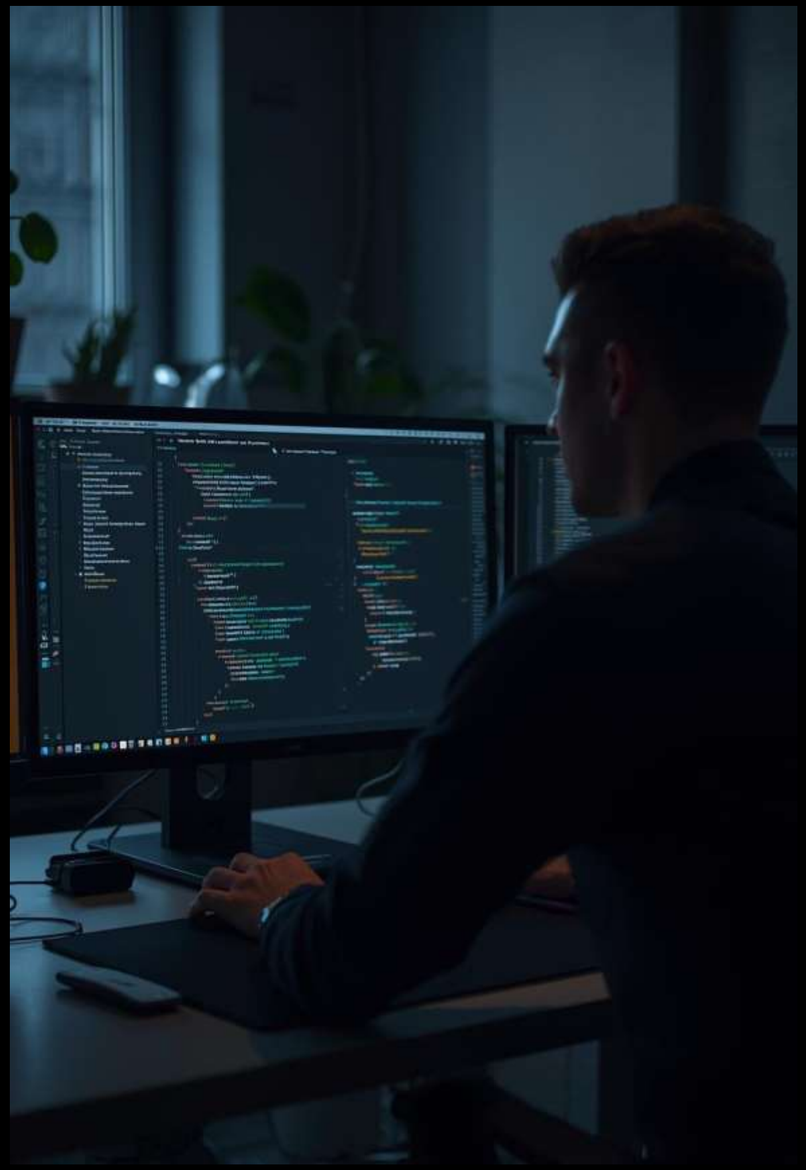
Why do we need a Builder?

Telescoping Constructors
Adding more PC parts means creating too many constructors. It becomes messy and hard to manage.

Parameter Confusion Long lists of parameters like (16, 512, 8, 2) are confusing. It's easy to swap the RAM with Storage by mistake.

Poor Readability new PC("i7", "16GB", "RTX", "512GB")Looking at this code, we cannot tell which value belongs to which component.

Maintenance Nightmare Adding a single new part (like a Power Supply) requires changing every constructor in the entire project.



PC Builder
Case Study

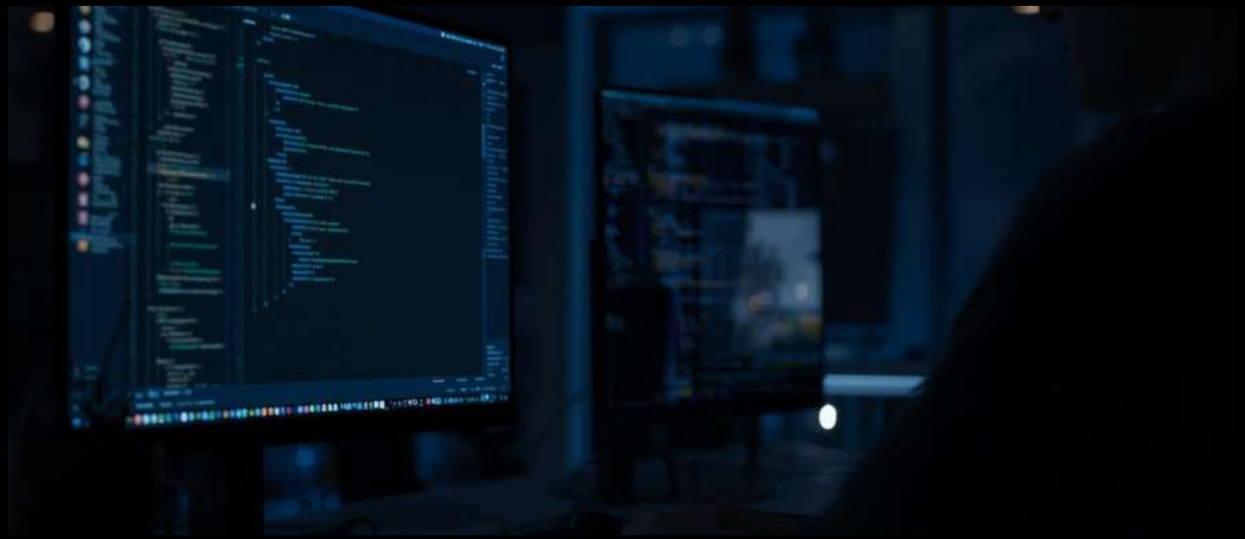
The Builder Pattern Solution

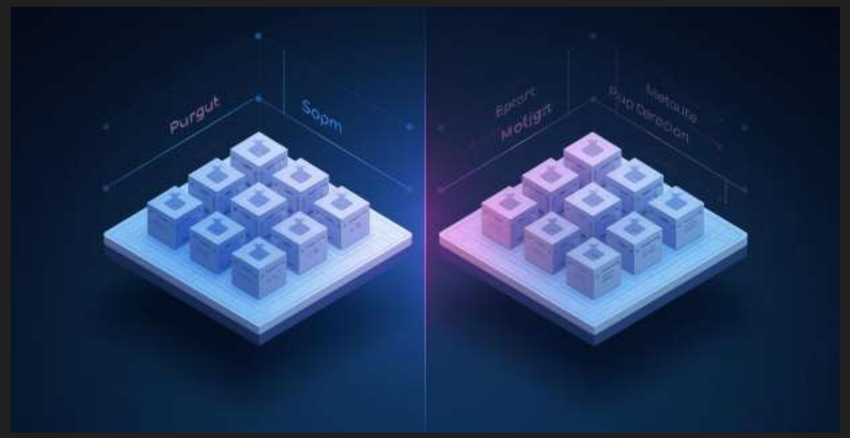
Build Step-by-Step ☐
We stop using giant constructors. Instead, we build the PC one part at a time.

Director Orchestration 🕴️💼
Instead of chaining methods, the PCDirector handles the exact building sequence. The client code is completely isolated from the complex assembly process!

Clean & Safe ☐
No more parameter confusion. We only add the components we actually need (No null values).

```
// 1. Build a Gaming PC
PCBuilder gamingBuilder = new GamingPCBuilder();
PCDirector director = new PCDirector(builder: gamingBuilder);
director.constructPC();
PC gamingPC = gamingBuilder.getResult();
System.out.println(x: "Building Gaming PC:");
gamingPC.showSpecs();
```





Builder vs Abstract Factory

Builder Pattern ☐
Focus: HOW the object is built (Step-by-step).

Return Timing: Returns the product only at the end (build()).

Use Case: Single, complex objects with many optional parts (Like our PC).

Abstract Factory 🏭
Focus: WHAT objects are created (Families of products).

Return Timing: Returns the product immediately.

Use Case: Creating families of related objects (Example: Windows UI vs. Mac UI components like Buttons and Checkboxes).

PC
Customization
System

System Architecture & UML Design

Builder Pattern

The Director 🤵💼

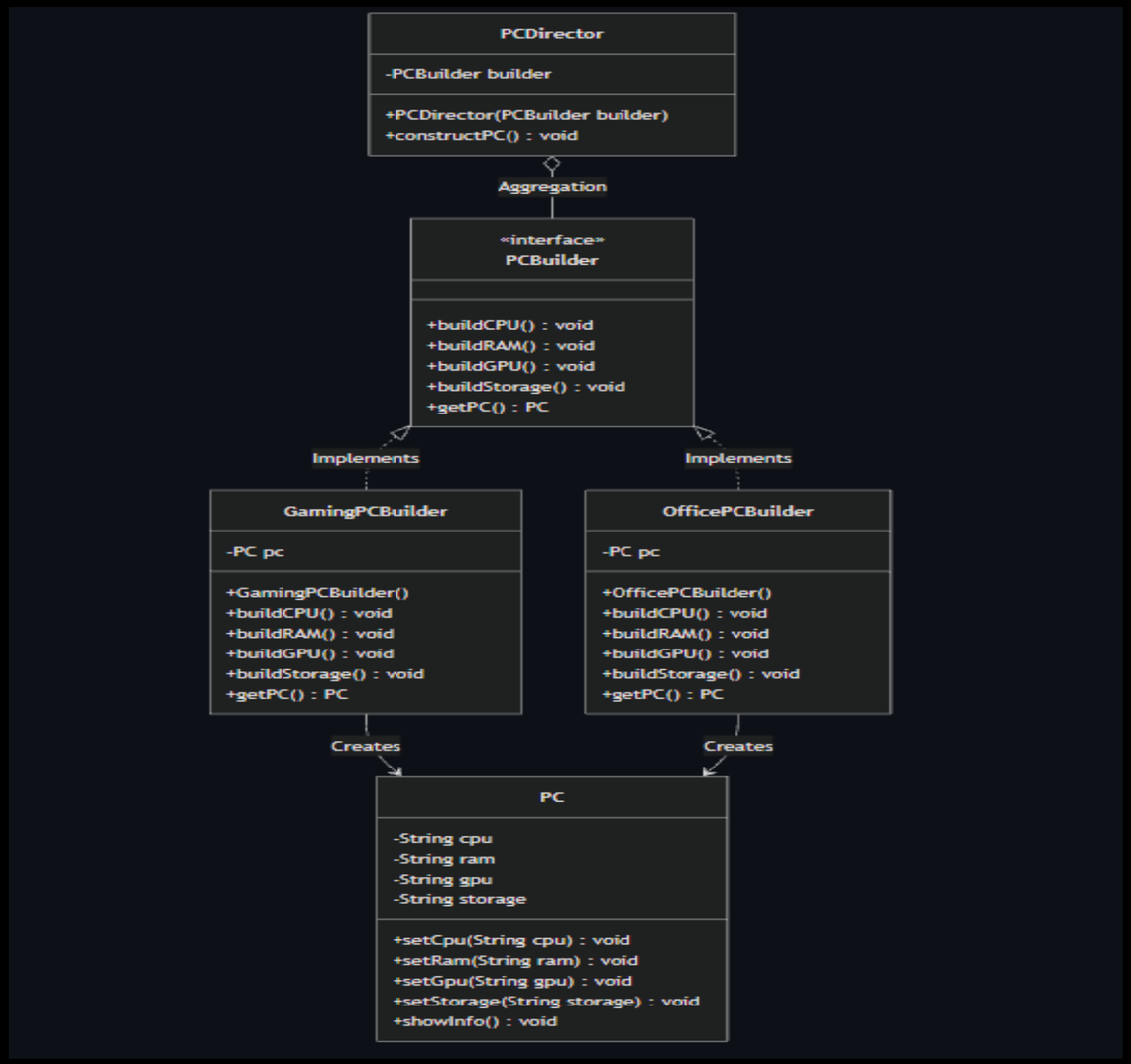
Orchestrates the building process. It knows the exact sequence to build a specific PC.

PCBuilder (Interface) 📋Standardizes the assembly steps (e.g., setCPU(), setGPU()).

Concrete Builders 👨‍🏭Implement the interface for specific configurations (GamingPCBuilder & OfficePCBuilder)

.

The Product 💻The final complex object (The PC), which is completely decoupled from the construction logic.



Java Implementation Details

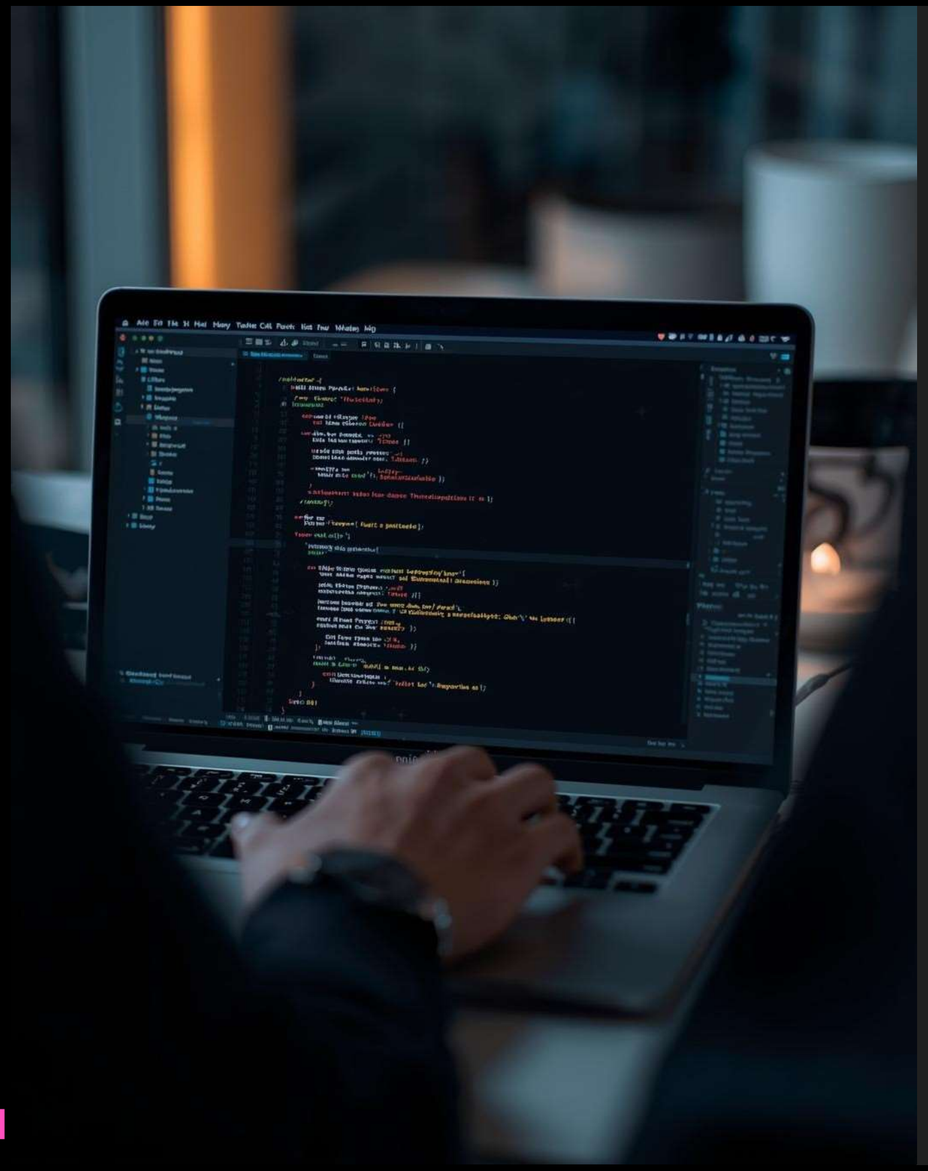
1. PCBuilder Interface 📋

Defines the blueprint. It forces all builders to implement methods like buildCPU() and buildGPU().

2. Concrete Builders 🧑‍🔧
Classes like GamingPCBuilder implement the interface to provide specific parts (e.g., setting an RTX 4090).
3. Encapsulating the Logic 🔒

The PCDirector hides the complexity.

```java
public interface PCBuilder {
    void buildCPU();
    void buildRAM();
    void buildGPU();
    void buildStorage();
    PC getResult();
}
```



Final Output

```
Building Gaming PC:
========= PC Configuration =========
CPU:     Intel Core i9-14900K
RAM:     32GB DDR5 6000MHz
GPU:     NVIDIA RTX 4090 24GB
Storage: 2TB NVMe Gen5 SSD
====================================
```

Thank You!

Builder
Pattern
Project